



StarStore فروشگاه

مبانی برنامه نویسی

استاد: دکتر تورج آرمین پور

اعضای گروه:

شایان رضوی: مهندسی عمران ۴۰۴۱۰۹۹۵۲

پرنده برک پور: مهندسی عمران ۴۰۴۱۰۴۷۲۷

سپهر ناظمی: مهندسی عمران ۴۰۴۱۰۵۳۰۴

نورین جعفری: مهندسی عمران ۴۰۴۱۰۴۷۸۴



مقدمه

این پروژه با هدف طراحی یک سیستم مدیریت فروشگاه به زبان برنامه‌نویسی پایتون و با استفاده از مفهوم شی‌گرایی (Object-Oriented Programming) یا به اختصار OOP و همچنین با استفاده از دیتابیس (MySQL) در محیط کنسول اجرا می‌شود. و تلاش شده است تا با بهره‌گیری از اصول طراحی نرم‌افزار، سیستمی کارآمد، منظم و قابل توسعه پیاده‌سازی گردد. استفاده از رویکرد شی‌گرایی در کنار پایگاه داده رابطه‌ای، امکان مدیریت بهینه داده‌ها و ارتباط بین بخش‌های مختلف برنامه را فراهم کرده است

هدف کلی

هدف اصلی از ایجاد این سیستم، طراحی نرم‌افزاری است که بتواند فرآیند مدیریت کالاها، ادمین‌ها و کلاینت‌ها را به‌طور بهینه و ساختاریافته انجام دهد. این سیستم نه تنها وظایف پایه‌ای مانند ثبت، ویرایش و حذف اطلاعات را پوشش می‌دهد، بلکه با رعایت اصول ارث‌بری (Inheritance) و چندریختی (Polymorphism) در برنامه‌نویسی شی‌گرا، بستری انعطاف‌پذیر برای گسترش و نگهداری آسان در آینده فراهم کرده است. به این ترتیب، امکان اعمال تغییرات در داده‌ها یا اضافه کردن قابلیت‌های جدید بدون نیاز به بازنویسی گسترده کد وجود خواهد داشت.

چکیده ی ساختار کد:

★ فایل starStore.py

فروشگاه متولد شده و فرآیند اجرای پروژه آغاز می‌گردد. این فایل شامل مقدمات راه‌اندازی اجزای مختلف پروژه و شروع تعامل با کاربر می‌باشد. فایل generated_data.py برای افزودن اطلاعات اولیه و داده‌های نمونه به پایگاه داده طراحی شده است تا محیطی شبیه‌سازی شده برای تست و نمایش عملکرد سیستم فراهم آورد. کلاس ShopDatabase در فایل tables.py مسئولیت مدیریت پایگاه داده را بر عهده دارد که در آن جداول مختلف ساخته شده و توابعی برای افزودن، به‌روزرسانی و دریافت آسان داده‌ها پیاده‌سازی شده است. در فایل designer.py کلاس کوچکی به همین نام برای تعامل با کاربر و ارائه گزارش‌های متنوع تعبیه شده است. همچنین فایل discount_codes.json شامل تعدادی کد تخفیف نمونه با فرمت JSON می‌باشد که این کدها نیز توسط فایل generated_data به پایگاه داده اضافه می‌شوند.

★ فایل classes.py

کلاس‌های اصلی پروژه شامل User، Admin، Client و Good یا product تعریف شده‌اند. کلاس User به عنوان کلاس پایه، ویژگی‌های مشترک کاربران مانند نام کاربری، رمز عبور، ایمیل و موجودی کیف پول را در خود جای داده است. کلاس Admin از User ارث‌بری کرده و برای مدیریت سطح مدیر سیستم طراحی شده و کلاس Client نیز که برای مشتریان است، از User مشتق می‌شود. کلاس Good نیز ساختار مربوط به محصولات شامل نام، قیمت، موجودی و سایر مشخصات را تعریف می‌کند.

★ فایل admin.py

توابع مورد نیاز برای پنل مدیریت پیاده‌سازی شده‌اند که شامل تابع signin برای ورود مدیر، تابع admin_menu برای نمایش منوی اصلی مدیریت، تابع add_good برای افزودن محصول جدید، تابع change_inventory برای تغییر موجودی محصولات و تابع change_price برای ویرایش قیمت کالاها می‌باشد. در فایل client.py نیز توابع مربوط به تعامل کاربران عادی یا مشتریان با سیستم قرار داده شده است. این توابع عبارتند از: get_time برای دریافت زمان جاری، client_start برای شروع کار با پنل کاربری، log_in برای ورود

به حساب کاربری، client_menu برای نمایش منوی مشتری، show_goods برای نمایش لیست محصولات، new_shopping برای ایجاد یک خرید جدید، history برای نمایش تاریخچه خریدها، change_password برای تغییر رمز عبور و charge_credit برای افزایش اعتبار کیف پول.

مشخصات فنی:

نقش‌های کاربری

در این سیستم دو نوع نقش کاربری تعریف شده است. ادمین کاربری با سطح دسترسی یک است که قابلیت تغییر ویژگی‌های کالاها و اضافه کردن کالا‌های جدید را دارد. مشتری، کاربری با سطح دسترسی دو است که میتواند اعتبار خود را افزایش دهد، اقدام به خرید کالا‌های موجود نماید و سوابق خرید‌های خود را مشاهده کند.

کنترل دسترسی و احراز هویت

در زمان ورود به سیستم نقش کاربر بر اساس تطابق نام کاربری و رمز عبور با سطح دسترسی موجود در پایگاه داده شناسایی شده و منوی مربوط به همان نقش برای کاربر نمایش داده میشود.

عملیات خرید کالا

مشتریان پس از ورود یا ثبت‌نام در سیستم میتوانند بر اساس موجودی کالاها و اعتبار خود اقدام به خرید نمایند. اطلاعات مربوط به هر خرید در جدول Histories ثبت شده و قابلیت بازبایی و مشاهده در آینده را دارد.

تغییر ویژگی یا اضافه کردن کالا

ادمین‌ها پس از ورود به سیستم قادر خواهند بود قیمت و موجودی هر یک از کالاها را تغییر دهند. همچنین امکان اضافه کردن کالای جدید با وارد کردن کد کالا و سایر ویژگی‌های مورد نیاز برای آنان فراهم است.

تغییر رمز عبور

تمامی کاربران میتوانند رمز عبور خود را تغییر دهند. رمز عبور جدید باید حداقل شامل ۸ کاراکتر بوده و ترکیبی از اعداد و حروف (کوچک و بزرگ) باشد

```
def create_tables(self):
    self.cursor.execute('''
        CREATE TABLE IF NOT EXISTS users (
            username TEXT PRIMARY KEY,
            password TEXT,
            access INTEGER
        )
    ''')
    self.cursor.execute('''
        CREATE TABLE IF NOT EXISTS Extrainfos (
            username TEXT PRIMARY KEY,
            credit INTEGER,
            FOREIGN KEY (username) REFERENCES users (username)
        )
    ''')
    self.cursor.execute('''
        CREATE TABLE IF NOT EXISTS goods (
            ocode TEXT PRIMARY KEY,
            price INTEGER,
            number INTEGER
        )
    ''')
    self.cursor.execute('''
        CREATE TABLE IF NOT EXISTS codes (
            code TEXT PRIMARY KEY,
            discount INTEGER,
            date TEXT,
            used INTEGER DEFAULT 1
        )
    ''')
    self.cursor.execute('''
        CREATE TABLE IF NOT EXISTS historis (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            username TEXT,
            orders TEXT,
            date TEXT,
            sum INTEGER,
            FOREIGN KEY (username) REFERENCES users (username)
        )
    ''')
    self.conn.commit()
```

تابع create_tables در صورت عدم وجود جداول مورد نیاز در پایگاه داده، اقدام به ایجاد آنها می‌کند. سپس تابع import_from_json فراخوانی شده و کدهای تخفیف را به پایگاه داده وارد می‌نماید.

```

def add_discount_code(self, code, discount, date, used=1):
    try:
        self.cursor.execute(
            "INSERT INTO codes (code, discount, date, used) VALUES (?, ?, ?, ?)",
            (code, discount, date, used)
        )
        self.conn.commit()
        return True
    except sqlite3.IntegrityError:
        return False

def import_from_json():
    with open('discount_codes.json', 'r') as file:
        discount_data = json.load(file)

    for code, details in discount_data.items():
        code_num = code[4:]
        expiry_date = details['expiry_date']
        discount = details['discount_percentage']
        db.add_discount_code(code_num, discount, expiry_date)

```

تابع `add_discount_code` کدهای تخفیف را وارد پایگاه داده می‌کند. این تابع داخل مازول `tables` قرار دارد. این مازول ما را از دانش دیتابیس بی‌نیاز میکند.

✦ تابع `start`

```

def start():
    if not is_inserted():
        generated_data.import_from_json()
        generated_data.authors_account()
        generated_data.sample()

    while True:
        try:
            D.greeting()
            order = D.safe_input("select your role:\n1- client\n2- admin")
            match order:
                case "1": client.client_start()
                case "2": admin.singin()
                case _:
                    D.error()
                    print("Invalid command. Please try again.\n")
        except:
            break

```

در ابتدای اجرای برنامه، وجود فایل نمونه شامل اطلاعات کاربران و کالاها و همچنین فایل کدهای تخفیف در پایگاه داده بررسی می‌شود. سپس نقش کاربر پرسیده شده و فرآیند ورود بر اساس آن نقش انجام می‌گیرد.

★ تابع `safe_input`

```
def safe_input(prompt):  
    prompt = prompt + "\nback\nquit\n"  
    user_input = input(prompt)  
    if user_input == "back":  
        raise Exception()  
    elif user_input == "quit":  
        exit()  
    return user_input
```

این تابع برای بازگشت به مرحله قبل و یا پایان دادن به برنامه تعریف شده است. با استفاده از این تابع، نیاز به بررسی مجدد این موارد در سایر توابع وجود ندارد.

★ فرآیند ورود کاربر

```
def log_in(x:int):  
    global user  
    while True:  
        try:  
            D.line()  
            data = D.safe_input("Enter your username and password: ").split()  
            if x == 1:  
                if db.verify_user(data[0], data[1], 2):  
                    user = client(data[0], data[1])  
                    D.success()  
                    client_menu()  
                else:  
                    D.failed()  
            else:  
                if client.sign_up(data[0], data[1]):  
                    user = client(data[0], data[1])  
                    D.success()  
                    client_menu()  
        except:  
            break
```

```
def singin():
    global user
    while True:
        try:
            D.line()
            data = D.safe_input("Enter your username and password: ").split()
            if db.verify_user(data[0], data[1], 1):
                user = admin(data[0], data[1])
                D.success()
                admin_menu()
            else:
                D.failed()
        except:
            break
```

```
def verify_user(self, username, password, access_level):
    self.cursor.execute(
        "SELECT password, access FROM users WHERE username = ?", (username,)
    )
    result = self.cursor.fetchone()
    if not result or result[0] != password or result[1] != access_level:
        return False
    return True
```

این تابع نام کاربری و رمز عبور را از کاربر دریافت کرده و وجود کاربری با این مشخصات را در پایگاه داده بررسی می‌کند. در صورت تأیید، اطلاعات کاربر در متغیر سراسری user ذخیره می‌شود تا در مراحل بعدی بتوان از ویژگی‌های تعریف شده در کلاس‌های مربوطه استفاده کرد.

✦ مدیریت درخواست‌های نقش‌های کاربری

```
def client_start():
    while True:
        try:
            D.line()
            order = D.safe_input("Choose an operation:\n1- Sign in\n2- Sign up")
            match order:
                case "1":
                    log_in(1)
                case "2":
                    log_in(0)
            break
        except:
            break
def client_menu():
    while True:
        try:
```

```

D.menu("client")
print(f"CREDIT : {user.credit}")
order = D.safe_input('''Choose operation:
1- new shopping \n 2- history \n 3- change password \n 4- change credit
''')

match order:
    case "1": new_shopping()
    case "2": history()
    case "3": change_password(user)
    case "4": charge_credit()

except:
    break

```

```

def admin_menu():
    while True:
        try:
            D.menu("admin")
            order = D.safe_input('''Choose operation:
1- add good \n 2- inventory management \n 3- price management \n 4-
change password
''')

            match order:
                case "1": add_good()
                case "2": change_inventory()
                case "3": change_price()
                case "4": change_password(user)

        except:
            break

```

این توابع وظیفه مدیریت درخواست‌ها و قابلیت‌های مربوط به هر نقش کاربری را بر عهده دارند. در این بخش، برای مشتریان میزان اعتبار آن‌ها نیز نمایش داده می‌شود.

✦ نمایش سوابق و کالاها

```
def get_all_products(self):
    self.cursor.execute("SELECT * FROM goods")
    return self.cursor.fetchall()
```

```
def show_goods():
    headers = ["ocode", "price", "num"]
    all_goods = db.get_all_products()
    table_data = [[i[0], i[1], i[2]] for i in all_goods]
    print(tb.tabulate(table_data, headers=headers, tablefmt="simple"))
    print()
```

```
def history():
    data = db.get_user_history(user.username)
    headers = ["name", "orders", "date", "sum"]
    display_data = [[d[1], d[2], d[3], d[4]] for d in data]
    print(tb.tabulate(display_data, headers=headers, tablefmt="simple"))
    print()
```

توابع `show_goods` و `history` عملکرد مشابهی دارند. هر دو تابع اطلاعات مربوط به سوابق خرید و کالاهای موجود در فروشگاه را از پایگاه داده استخراج کرده و با استفاده از کتابخانه `tabulate`، این اطلاعات را به صورت جدولی در کنسول به کاربر نمایش می‌دهند.

✦ بررسی قدرت رمز عبور

تابع `password_identifier` برای بررسی میزان امنیت و قدرت رمز عبور ورودی مشتری طراحی شده است. این تابع یک مقدار بولین را به تابع `change_password` بازمی‌گرداند تا در صورت تأیید شرایط، رمز جدید در پایگاه داده ذخیره شود.

```
def update_user_password(self, username, new_password):
    self.cursor.execute(
        "UPDATE users SET password = ? WHERE username = ?", (new_password,
username)
    )
    self.conn.commit()
```

```
def change_password(self, order):
    if not self.password_identifier(order):
        print("This password is not strong enough.")
        D.failed()
        return False
    self.password = order
    db.update_user_password(self.username, order)
    return True
```

```
def change_password(x:client):
    while True:
        try:
            D.line()
            order = D.safe_input("Enter your new password:")
            if not x.change_password(order):
                continue
            D.success(end=True)
        except:
            break
```

```
def password_identifier(password):
    errors = []
    if len(password) < 8:
        errors.append("Please use at least 8 characters with mix of letters
and numbers.")
    has_upper = has_lower = has_digit = False
    for char in password:
        if 'A' <= char <= 'Z': has_upper = True
        elif 'a' <= char <= 'z': has_lower = True
        elif '0' <= char <= '9': has_digit = True
    if not has_upper: errors.append("Password must contain at least one
uppercase letter")
    if not has_lower: errors.append("Password must contain at least one
lowercase letter")
    if not has_digit: errors.append("Password must contain at least one
digit")

    if errors:
        print("\n".join(errors))
        D.failed()
        return False

    return True
```

✦ افزایش اعتبار مشتری

تابع `charge_credit` اعتبار فعلی مشتری را دریافت کرده و مقدار درخواستی را به آن اضافه می‌نماید. سپس نتیجه نهایی در پایگاه داده ذخیره می‌شود.

```
def new_shopping():
    while True:
        try:
            D.line()
            show_goods()
            order1 = D.safe_input("Enter product codes and quantities :")
            x = order1.split(' ')
            if len(x) % 2 != 0:
                continue
            goods = [good(x[i]) for i in range(len(x)) if i%2==0]
            nums = [int(x[i]) for i in range(len(x)) if i%2==1]
            is_exist = True
            total = 0
            for k in range(len(goods)):
                total += goods[k].price * nums[k]
                if goods[k].number < nums[k]:
                    is_exist = False
                    print(f"{goods[k].ocode} not enough stock available")
            if not is_exist:
                D.failed()
                continue
            elif total > user.credit:
                print("You dont have enough credit.")
                D.failed()
                continue
            order2 = input(f"Total value of shopping is :{total}\nDo you wish to proceed? ?\n1- Yes\n2- No\n")
            if order2 == "2":
                break
            order2 = input("Do you have a discount code ?\n1- Yes\n2- No\n")
            discount = 0
            if order2 == "1":
                code = input("Enter your code:")
                code_info = db.get_discount_code(code)
                if not code_info or code_info[0] == 0 or code_info[1] <
get_time():
                print("Invalid or expired discount code")
                D.failed()
                continue
```

```

        discount = code_info[2]
        db.use_discount_code(code)

        user.change_credit(-int(0.95*(total - discount)))
        db.add_history(user.username, order1, get_time(), total - discount)
        for l in range(len(goods)):
            goods[l].change_number(-nums[l])
        D.success(end=True)
    except:
        break
def charge_credit():
    while True:
        try:
            D.line()
            order = D.safe_input("Enter the amount you wish to charge: ")
            user.change_credit(int(order))
            D.success(end=True)
        except:
            break

```

با وجود پیچیدگی ظاهری، این تابع نقش مهمی در پیوند دادن بخش های مختلف از جمله کدهای تخفیف، اتریبیوت های کاربری و همچنین تغییر ویژگی کالاها دارد. ابتدا اُکد کالاها و تعداد هر کدام توسط مشتری وارد می شود. سپس با بررسی موجودی هر کالا و مجموع نهایی خرید، از کاربر پرسیده می شود که قصد خرید دارد یا خیر و در صورت داشتن کد تخفیف آن را وارد کند. سپس تغییرات مربوط به موجودی هر کالا، اعتبار مشتری، کد استفاده شده و همچنین سوابق خرید انجام می شود.

این توابع وظیفه اعمال تغییرات داده شده توسط ادمین در پایگاه داده را برعهده دارند.

```

def change_price():
    while True:
        try:
            D.line()
            order = D.safe_input("Enter ocode and changed price: ").split(' ')
            if len(order) != 2:
                continue
            my_good = good(order[0])
            if my_good.exists:
                my_good.change_price(int(order[1]))
                D.success(end=True)
            else:
                print("Product not found")
                D.failed()
        except:
            break

```

تابع `change_price` برای به‌روزرسانی قیمت کالاها در سیستم فروشگاه طراحی شده است. این تابع با دریافت کد محصول و قیمت جدید از کاربر، ابتدا وجود کالای مورد نظر را در پایگاه داده بررسی می‌کند. در صورت یافتن کالا، قیمت آن بر اساس مقدار وارد شده به‌روزرسانی شده و پیام موفقیت عملیات به کاربر نمایش داده می‌شود. چنانچه کالایی با کد وارد شده یافت نشود، پیغام خطای مربوطه نمایش داده خواهد شد. این فرآیند تا زمانی که ورودی صحیح دریافت شود یا خطایی رخ دهد ادامه پیدا می‌کند.

✦ تابع `add_good`

```
def add_good():
    while True:
        try:
            D.line()
            order = D.safe_input("Enter product details : ( Code, Price,
Quantity) ").split(' ')
            if len(order) != 3:
                continue
            my_good = good(order[0])
            if my_good.exists:
                print("Product with this code already exists")
                D.failed()
            else:
                db.add_product(order[0], int(order[1]), int(order[2]))
                D.success(end=True)
        except:
            break
```

تابع `add_good` برای افزودن محصولات جدید به سیستم فروشگاه طراحی شده است. این تابع ابتدا کد، قیمت و تعداد محصول را از کاربر دریافت کرده و سپس بررسی می‌کند که آیا محصولی با این کد از قبل در پایگاه داده وجود دارد یا خیر. در صورت تکراری بودن کد محصول، پیغام خطا نمایش داده شده و عملیات افزودن محصول متوقف می‌شود. چنانچه کد محصول یکتا باشد، محصول جدید با مشخصات وارد شده به پایگاه داده اضافه می‌گردد. این فرآیند تا زمانی که ورودی صحیح دریافت شود ادامه می‌یابد و در صورت بروز هرگونه خطا، اجرای حلقه متوقف خواهد شد.

✦ تابع `change_inventory`

```
def change_inventory():
    while True:
        try:
            D.line()
            order = D.safe_input("Enter ocode and changed inventory:").split(' ')
            if len(order) != 2:
                continue
            my_good = good(order[0])
```

```

    if my_good.exists:
        my_good.change_number(int(order[1]))
        D.success(end=True)
    else:
        print("Product not found")
        D.failed()
except:
    break

```

تابع `change_inventory` برای به‌روزرسانی موجودی کالاها در سیستم فروشگاه طراحی شده است. این تابع با دریافت کد محصول و مقدار تغییر موجودی از کاربر، ابتدا وجود کالای مورد نظر را در پایگاه داده بررسی می‌کند. در صورت یافتن کالا، موجودی آن بر اساس مقدار درخواستی به‌روزرسانی شده و پیام موفقیت عملیات به کاربر نمایش داده می‌شود. چنانچه کالایی با کد وارد شده یافت نشود، پیغام خطای مربوطه نمایش داده خواهد شد. این فرآیند تا زمانی که ورودی صحیح دریافت شود یا خطایی رخ دهد ادامه پیدا می‌کند. از این قابلیت برای مدیریت و کنترل موجودی انبار استفاده می‌شود.

✦ کلاس admin

```

class admin(User):
    def __init__(self, username, password):
        super().__init__(username, password)

```

این کلاس که از کلاس اصلی `User` ارث‌بری کرده است، برای تعریف کاربران با سطح دسترسی مدیر طراحی شده است. در زمان ایجاد یک شی از این کلاس، نام کاربری و رمز عبور دریافت شده و با استفاده از تابع سازنده کلاس والد، عملیات مقداردهی اولیه انجام می‌گیرد. این ساختار امکان توسعه قابلیت‌های مختص مدیران را در آینده فراهم می‌کند.

✦ کلاس client

```

class client(User):
    def __init__(self, username, password):
        super().__init__(username, password)
        self.credit = db.get_credit(username)

    def change_credit(self, order):
        self.credit += int(order)
        db.update_credit(self.username, int(order))

    @staticmethod
    def sign_up(username, password):
        if db.user_exists(username):
            print("This username already exist.")
            D.failed()

```

```

        return False
    if not client.password_identifier(password):
        print("This password is not strong enough.")
        D.failed()
        return False
    db.add_user(username, password, 2)
    db.add_extrainfo(username, 0)
    return True

```

این کلاس که از کلاس User مشتق شده است، نماینده کاربران عادی سیستم می‌باشد. در زمان ایجاد یک شی client، علاوه بر ثبت نام کاربری و رمز عبور، میزان اعتبار کاربر نیز از پایگاه داده دریافت و در ویژگی credit ذخیره می‌شود. متد change_credit برای افزایش یا کاهش موجودی اعتبار کاربر استفاده می‌شود و همزمان مقدار credit در شی و پایگاه داده به‌روزرسانی می‌گردد. متد ایستای sign_up مسئولیت ثبت‌نام کاربران جدید را بر عهده دارد. این متد ابتدا بررسی می‌کند که نام کاربری تکراری نباشد، سپس قدرت رمز عبور را ارزیابی می‌کند و در صورت تأیید، اطلاعات کاربر را با سطح دسترسی ۲ (کاربر عادی) و اعتبار اولیه صفر در پایگاه داده ذخیره می‌نماید.

✦ کلاس good

```

class good:
    def __init__(self, ocode):
        self.ocode = ocode
        product = db.get_product(ocode)
        if product:
            self.price, self.number = product
            self.exists = True
        else:
            self.price = self.number = 0
            self.exists = False

    def change_number(self, order):
        self.number += order
        db.update_product_stock(self.ocode, order)

    def change_price(self, order):
        self.price = order
        db.update_product_price(self.ocode, order)

```

این کلاس برای مدیریت و نمایش اطلاعات کالاها در سیستم طراحی شده است. هنگام ایجاد یک شی از این کلاس، کد کالا دریافت شده و اطلاعات مربوط به آن شامل قیمت و موجودی از پایگاه داده استخراج می‌گردد. در صورت وجود کالا، ویژگی exists به True تنظیم شده و اطلاعات تکمیل می‌شود، در غیر این صورت exists برابر False خواهد بود. متد change_number برای تغییر موجودی کالا استفاده می‌شود که همزمان مقدار موجودی در شی و پایگاه داده به‌روزرسانی می‌گردد. متد change_price نیز همین وظیفه را برای تغییر قیمت کالا بر عهده دارد و پس از دریافت قیمت جدید، آن را در شی و پایگاه داده ثبت می‌کند.

```

import json
from tables import ShopDatabase

db = ShopDatabase()

def import_from_json():
    with open('discount_codes.json', 'r') as file:
        discount_data = json.load(file)

    for code, details in discount_data.items():
        code_num = code[4:]
        expiry_date = details['expiry_date']
        discount = details['discount_percentage']
        db.add_discount_code(code_num, discount, expiry_date)

def authors_account():
    authors = [('Sepehr', 'sepehr1385', 1), ('Shayan', 'shayan1386', 1),
               ('Noorin', 'noorin1385', 1), ('Parand', 'parand1385', 1)]

    for username, password, access in authors:
        db.add_user(username, password, access)

def sample():
    users = [('admin', 'admin123', 1), ('ali', 'Ali12345', 2),
             ('sara', 'Sara12345', 2)]

    for username, password, access in users:
        db.add_user(username, password, access)
    db.add_extrainfo('ali', 500000)
    db.add_extrainfo('sara', 300000)

    products = [('L001', 35000000, 10), ('M001', 500000, 50),
                 ('K001', 2000000, 20), ('H001', 1500000, 15)]

    for ocode, price, number in products:
        db.add_product(ocode, price, number)

```

Generated_data یک ماژول نمونه است که برای وارد کردن اطلاعات اولیه به پایگاه داده طراحی شده است. این ماژول شامل سه تابع اصلی می‌باشد که هر یک وظایف مشخصی را بر عهده دارند. تابع import_from_json ابتدا اطلاعات فایل json حاوی کدهای تخفیف را در یک دیکشنری ذخیره کرده، سپس مقادیر مربوط به هر کد تخفیف و تاریخ انقضای آن را استخراج نموده و در نهایت این کدها را به پایگاه داده اضافه می‌کند. تابع Authors_account برای ایجاد حساب کاربری نویسندگان برنامه در نظر گرفته شده است که برای هر یک از آنها نام کاربری و

رمز عبور متناظر با سطح دسترسی مدیر تعریف می‌گردد. تابع Sample نیز مسئولیت وارد کردن تعدادی کاربر با سطوح دسترسی مدیر و کلاینت به همراه تعدادی کالای نمونه را بر عهده دارد.

✦ کلاس designer

```
class designer:

    @staticmethod
    def safe_input(prompt):
        prompt = prompt + "\nback\nquit\n"
        user_input = input(prompt)
        if user_input == "back":
            raise Exception()
        elif user_input == "quit":
            exit()
        return user_input

    def greeting():
        print("----- Welcome to StarStore ☑ ----- \n")

    def menu(x):
        x = x.title()
        if x == "Admin":
            print(f"----- {x} Menu 👑 ----- \n")
        else:
            print(f"----- {x} Menu 📁 ----- \n")

    def line():
        print("----- \n")

    def success(end=False):
        print("----- The Operation was successfull ✔ ----- \n")
        if end : raise Exception()

    def failed():
        print("----- The Operation failed ✖ ----- \n")

    def error():
        print("----- ! Error ! ----- \n")
```

کلاس designer به منظور مدیریت پیام‌ها و خطاهای نمایش داده شده به کاربر طراحی شده است و شامل متدهایی می‌باشد که هر یک متناسب با فعالیت کاربر، پیام‌های مشخصی را نمایش می‌دهند. متد safe_input وظیفه کنترل بازگشت یا خاتمه برنامه را بر عهده دارد، به گونه‌ای که در هر مرحله از اجرا، اگر کاربر عبارت back را ارسال کند، یک سطح به عقب بازگشته و اگر quit را وارد کند، برنامه به طور کامل

متوقف می‌شود. متد `greeting` در آغاز اجرای برنامه فراخوانی شده و برای خوشامدگویی به کاربر مورد استفاده قرار می‌گیرد. متد `menu` صفحه اصلی برنامه را با توجه به نقش کاربر نمایش داده و پیام‌های متفاوتی را چاپ می‌کند. متد `line` در بخش‌هایی از کد برای زیباسازی خروجی به کار گرفته شده است. متد `success` زمانی اجرا می‌شود که عملیات انجام شده توسط کاربر یا مدیر با موفقیت به پایان رسیده و پیامی مبنی بر تأیید عملیات چاپ می‌کند. متد `failed` در شرایطی که عملیات با شکست مواجه شود، پیام خطای مربوطه را نمایش می‌دهد. متد `error` نیز هنگامی که کاربر گزینه‌ای نامعتبر ارسال کند، پیام خطا را چاپ می‌کند.

✦ ماژول tables

```
import sqlite3

class ShopDatabase:

    def __init__(self, db_name='shop.db'):
        self.db_name = db_name
        self.connect()
        self.create_tables()

    def connect(self):
        self.conn = sqlite3.connect(self.db_name)
        self.cursor = self.conn.cursor()

    def get_db(self):
        return self.conn, self.cursor

    def close(self):
        if self.conn:
            self.conn.close()

    def create_tables(self):
        self.cursor.execute('''
            CREATE TABLE IF NOT EXISTS users (
                username TEXT PRIMARY KEY,
                password TEXT,
                access INTEGER
            )
        ''')
        self.cursor.execute('''
            CREATE TABLE IF NOT EXISTS Extrainfos (
                username TEXT PRIMARY KEY,
                credit INTEGER,
                FOREIGN KEY (username) REFERENCES users (username)
            )
        ''')
        self.cursor.execute('''
```

```

        CREATE TABLE IF NOT EXISTS goods (
            ocode TEXT PRIMARY KEY,
            price INTEGER,
            number INTEGER
        )
    '''
    self.cursor.execute('''
        CREATE TABLE IF NOT EXISTS codes (
            code TEXT PRIMARY KEY,
            discount INTEGER,
            date TEXT,
            used INTEGER DEFAULT 1
        )
    ''')
    self.cursor.execute('''
        CREATE TABLE IF NOT EXISTS historis (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            username TEXT,
            orders TEXT,
            date TEXT,
            sum INTEGER,
            FOREIGN KEY (username) REFERENCES users (username)
        )
    ''')
    self.conn.commit()

def add_user(self, username, password, access=2):
    try:
        self.cursor.execute(
            "INSERT INTO users (username, password, access) VALUES (?, ?, ?)",
            (username, password, access)
        )
        self.conn.commit()
        return True
    except sqlite3.IntegrityError:
        return False

def add_extrainfo(self, username, credit=0):
    try:
        self.cursor.execute(
            "INSERT INTO Extrainfos (username, credit) VALUES (?, ?)",
            (username, credit)
        )
        self.conn.commit()

```

```

        return True
    except sqlite3.IntegrityError:
        return False

    def update_user_password(self, username, new_password):
        self.cursor.execute(
            "UPDATE users SET password = ? WHERE username = ?", (new_password,
username)
        )
        self.conn.commit()

    def get_credit(self, username):
        self.cursor.execute(
            "SELECT credit FROM Extrainfos WHERE username = ?", (username,)
        )
        result = self.cursor.fetchone()
        return result[0] if result else 0

    def update_credit(self, username, amount):
        self.cursor.execute(
            "UPDATE Extrainfos SET credit = credit + ? WHERE username = ?",
(amount, username)
        )
        self.conn.commit()

    def add_product(self, ocode, price, number):
        try:
            self.cursor.execute(
                "INSERT INTO goods (ocode, price, number) VALUES (?, ?, ?)",
(ocode, price, number)
            )
            self.conn.commit()
            return True
        except sqlite3.IntegrityError:
            return False

    def update_product_price(self, ocode, new_price):
        self.cursor.execute(
            "UPDATE goods SET price = ? WHERE ocode = ?", (new_price, ocode)
        )
        self.conn.commit()

    def update_product_stock(self, ocode, change):
        self.cursor.execute(

```

```

        "UPDATE goods SET number = number + ? WHERE ocode = ?", (change,
ocode)
    )
    self.conn.commit()

def get_product(self, ocode):
    self.cursor.execute(
        "SELECT price, number FROM goods WHERE ocode = ?", (ocode,)
    )
    return self.cursor.fetchone()

def get_all_products(self):
    self.cursor.execute("SELECT * FROM goods")
    return self.cursor.fetchall()

def add_discount_code(self, code, discount, date, used=1):
    try:
        self.cursor.execute(
            "INSERT INTO codes (code, discount, date, used) VALUES (?, ?, ?,
?),",
            (code, discount, date, used)
        )
        self.conn.commit()
        return True
    except sqlite3.IntegrityError:
        return False

def get_discount_code(self, code):
    self.cursor.execute(
        "SELECT used, date, discount FROM codes WHERE code = ?", (code,)
    )
    return self.cursor.fetchone()

def use_discount_code(self, code):
    self.cursor.execute(
        "UPDATE codes SET used = 0 WHERE code = ?", (code,)
    )
    self.conn.commit()

def add_history(self, username, orders, date, total_sum):
    self.cursor.execute(
        "INSERT INTO historis (username, orders, date, sum) VALUES (?, ?, ?,
?),",
        (username, orders, date, total_sum)
    )

```

```

        self.conn.commit()
        return self.cursor.lastrowid

    def get_user_history(self, username):
        self.cursor.execute(
            "SELECT * FROM historis WHERE username = ? ORDER BY date DESC",
            (username,)
        )
        return self.cursor.fetchall()

    def user_exists(self, username):
        self.cursor.execute(
            "SELECT * FROM users WHERE username = ?", (username,)
        )
        return self.cursor.fetchone() is not None

    def verify_user(self, username, password, access_level):
        self.cursor.execute(
            "SELECT password, access FROM users WHERE username = ?", (username,)
        )
        result = self.cursor.fetchone()
        if not result or result[0] != password or result[1] != access_level:
            return False
        return True

```

در فایل `tables.py`، کلاسی به نام `ShopDatabase` طراحی و پیاده‌سازی شده است که هدف اصلی آن ساده‌سازی و یکپارچه‌سازی تعامل با پایگاه داده می‌باشد. بخش اول متدهای این کلاس شامل `__init__` به عنوان سازنده، `connect` برای برقراری اتصال به پایگاه داده، `get_db` برای دریافت شیء پایگاه داده و `close` برای بستن اتصال است که همگی مختص راه‌اندازی اولیه پایگاه داده و مدیریت اتصال به آن هستند. همچنین متدی به نام `create_tables` در این کلاس در نظر گرفته شده که وظیفه آن ساخت جدول‌های مورد نیاز در پایگاه داده مطابق با خواسته‌های پروژه می‌باشد. این جدول‌ها شامل `users` برای ذخیره اطلاعات کاربران، `Extrainfos` برای نگهداری اطلاعات تکمیلی، `goods` برای مدیریت محصولات، `codes` برای کدهای تخفیف یا تأیید و `histories` برای ثبت تاریخچه خریدها و فعالیت‌های کاربران می‌باشند. در انتها این کلاس شامل متدهای متعددی عمدتاً با عناوین `add`، `update` و `get` است که برای تغییر، اضافه کردن و گرفتن ساده‌تر انواع داده‌ها برای جداول مختلف پایگاه داده تهیه شده است. این متدها با انتزاع عملیات پیچیده پایگاه داده، امکان انجام عملیات رایج مانند افزودن کاربر جدید، به‌روزرسانی موجودی کالا، دریافت تاریخچه خرید یک مشتری یا استخراج اطلاعات محصولات را به شکلی ساده و خوانا فراهم می‌کنند. بدین ترتیب، سایر بخش‌های برنامه بدون نیاز به نوشتن مستقیم کوئری‌های `SQL` و با صرفاً فراخوانی این متدها می‌توانند با پایگاه داده تعامل داشته باشند که این امر باعث کاهش خطاها، افزایش امنیت و تسریع فرآیند توسعه نرم‌افزار می‌گردد.

✦ پیام‌های خطا

Incorrect username or password.	نام کاربری وارد شده در پایگاه داده وجود ندارد یا اینکه رمز وارد شده اشتباه است.
You don't have access.	نام کاربری و رمز وارد شده درست است اما شما نقش خود را اشتباه وارد کرده‌اید.
This password is not strong enough.	رمز عبور وارد شده به هنگام تغییر رمز یا ثبت نام، واجد شرایط نیست.
You don't have enough credit.	شما برای خرید اجناس به اندازه کافی موجودی ندارید.
Invalid or expired discount code.	تاریخ استفاده از این کد گذشته و یا ثبت نشده است.
[The product] not enough stock available.	تعداد انتخاب شده برای یکی از کالاهای مورد نظر از سقف موجودی آن بیشتر است.